

Write a C program to stimulate following contiguous memory allocation technique.

a. Worst fit b. Best fit c. First-fit.

```
#include <stdio.h>
```

```
#define MAX 100
```

```
void firstFit(int blocks[], int m, int processes[], int n);
```

```
void bestFit(int blocks[], int m, int processes[], int n);
```

```
void worstFit(int blocks[], int m, int processes[], int n);
```

```
void printResult(const char *title, int processes[], int allocation  
int n){
```

```
int allocation[MAX];
```

```
for(int i=0; i<n; i++) allocation[i] = -1;
```

```
for(int i=0; i<n; i++){
```

```
for(int j=0; j<m; j++){
```

```
if(blocks[j] >= processes[i]){
```

```
allocation[i] = j;
```

```
blocks[j] = 0;
```

```
break;
```

```
}
```

```
}
```

```
}
```

```
printf("\n%s\n", title);
```

```
printf("Proces No.\t Proces Size\t Block No.\n");
```

```
for(int i=0; i<n; i++){
```

```
printf("%d\t\t %d\t\t", i+1, processes[i]);
```

```
if(allocation[i] != -1)
```

```
printf("%d\n", allocation[i]+1);
```

```

else
    printf("Not Allocated\n");
}
}

void firstFit(int blocks[], int m, int process[], int n) {
    int allocation[MAX];
    for (int i = 0; i < n; i++) allocation[i] = -1;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (blocks[j] >= process[i]) {
                allocation[i] = j;
                blocks[j] = 0;
                break;
            }
        }
    }
    printResult("First fit Allocation:", process, allocation, n);
}

```

```

void bestFit(int blocks[], int m, int process[], int n) {
    int allocation[MAX];
    for (int i = 0; i < n; i++) allocation[i] = -1;
    for (int i = 0; i < n; i++) {
        int bestIdx = -1;
        for (int j = 0; j < m; j++) {
            if (blocks[j] >= process[i]) {
                if (bestIdx == -1 || blocks[j] < blocks[bestIdx])
                    bestIdx = j;
            }
        }
    }
}

```



```

        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            blocks[bestIdx] = 0;
        }
    }

    printResult("Best Fit Allocation:", processes, allocation, n);
}

```

```

void worstFit(int blocks[], int m, int processes[], int n) {
    int allocation[MAX];
    for (int i = 0; i < n; i++) allocation[i] = -1;
    for (int i = 0; i < n; i++) {
        int worstIdx = -1;
        for (int j = 0; j < m; j++) {
            if (blocks[j] >= processes[i]) {
                if (worstIdx == -1 || blocks[j] > blocks[worstIdx])
                    worstIdx = j;
            }
        }
        if (worstIdx != -1) {
            allocation[i] = worstIdx;
            blocks[worstIdx] = 0;
        }
    }

    printResult("Worst Fit Allocation:", processes, allocation, n);
}

```

```

int main() {
    int blocks[MAX], processes[MAX];
    int m, n;
}

```

```
printf("Enter number of memory blocks: ");
scanf("%d", &m);
```

```
printf("Enter sizes of blocks: \n");
for(int i=0; i<m; i++) scanf("%d", &blocks[i]);
```

```
printf("Enter number of processes: ");
scanf("%d", &n);
```

```
printf("Enter sizes of processes: \n");
for(int i=0; i<n; i++) scanf("%d", &processes[i]);
```

```
int blocks1[MAX], blocks2[MAX], blocks3[MAX];
```

```
for (int i=0; i<m; i++){
```

```
    blocks1[i] = blocks[i];
```

```
    blocks2[i] = blocks[i];
```

```
    blocks3[i] = blocks[i];
```

```
}
```

```
firstFit(blocks1, m, processes, n);
```

```
bestFit(blocks2, m, processes, n);
```

```
worstFit(blocks3, m, processes, n);
```

```
return 0;
```

```
}
```

Idx]];

output :

Enter number of memory blocks : 6

Enter sizes of blocks :

300 600 350 200 750 125

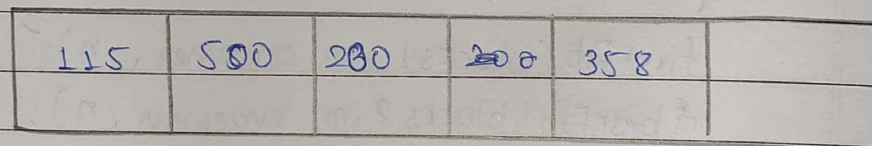
Enter number of processes : 5

Enter sizes of processes :

125 500 358 200 375

First fit Allocation:

Process No.	Process Size	Block No.
1	125	1
2	500	2
3	358	5
4	200	3
5	375	Not Allocated.

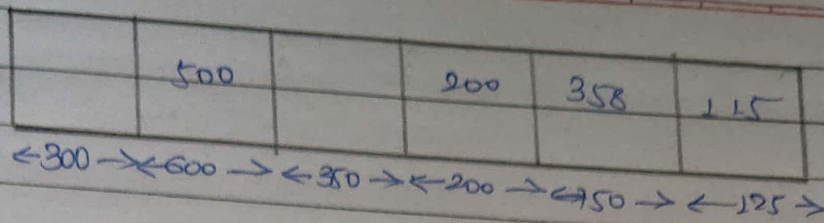


← 300 → × 600 → × 350 → × 200 → × 750 → × 125 →

375 is not allocated.

Best Fit Allocation:

Process No.	Process size	Block No.
1	125	6
2	500	2
3	358	5
4	200	4
5	375	Not allocated.



375 is not allocated.

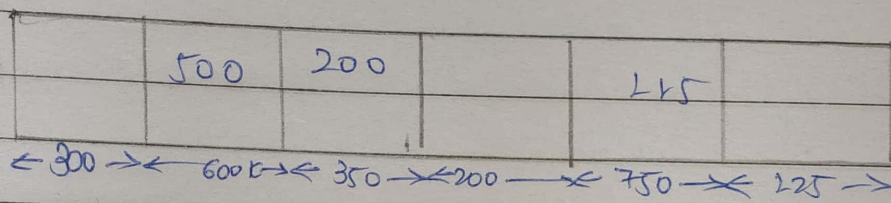
Worst Fit Allocation:

Process No.

Process Size

Block No.

1	115	5
2	500	2
3	358	Not Allocated
4	200	3
5	375	Not Allocated.



358 is not allocated.
375 is not allocated.

12/05/26